Langration 4차 발표 대본

- 발표: 최우석 · 2026-11-05 15:00
- 목표 시간: 약 14~15분 (시연 포함)
- 말하는 분량은 공백 빼고 약 2,900자입니다. 1분에 250자 정도로 또박또박 읽으면 약 12분입니다.
- 여기에 시연 중 답을 기다리는 시간과 화면 전환을 더해 14~15분이 되도록 맞췄습니다.
- 시간이 남으면 4장(3차 보완점)과 13장(지표별)에서 천천히 말하고, 모자라면 13장 설명을 줄이세요.
- ( ) 안은 행동 지시이고, 읽지 않습니다.

#	슬라이드	시간	누적
1	팀 소개	0:40	0:40
2	표지	0:30	1:10
3	발표 순서	0:30	1:40
4	3차 보완점	1:20	3:00
5	해결 방향	1:00	4:00
6	3차 → 4차, 바뀐 것만	0:50	4:50
7	배포 구성	1:10	6:00
8	Django API	1:10	7:10
9	질문 분류	1:00	8:10
10	진행 표시	1:00	9:10
11	시연	2:40	11:50
12	결과 정리	1:00	12:50
13	지표별 비교	1:00	13:50
14	RAG 평가	1:00	14:50
15	마무리	0:20	15:10

---

1. 팀 소개 (0:40)

안녕하세요, 4팀 발표를 맡은 최우석입니다. 먼저 팀원과 역할부터 소개하겠습니다.

저는 Django 프로젝트 폴더 구조를 설계하고, 백엔드 구현과 API 엔드포인트 정의, 그리고 발표를 맡았습니다. 조현주 님은 AWS와 Docker 배포 환경을 구축하고 LLM 파이프라인을 검증했습니다. 손채영 님은 LLM 파이프라인 검증과 QA를 맡았고, 심성욱 님은 화면 UI 구현과 백엔드 API 연결 보조, 그리고 지금 보시는 발표자료를 만들었습니다.

2. 표지 (0:30)

저희 서비스 이름은 Langration입니다. 옛 LangChain 코드를 최신 버전으로 고치고, 무엇을 근거로 고쳤는지, 검사는 통과했는지까지 보여 주는 웹 서비스입니다. 이번 4차에서는 Django, Docker Compose, AWS EC2로 실제 배포까지 했습니다.

3. 발표 순서 (0:30)

Langration은 3차 때 만든 챗봇을 바탕으로 합니다. 그래서 오늘은 3차와 같은 부분은 짧게 넘기고, 4차에서 바뀐 것 위주로 말씀드리겠습니다. 3차의 보완점, 해결 방향, 바뀐 기술을 설명한 뒤 바로 시연을 보여 드리고, 마지막에 결과를 정리하겠습니다.

4. 3차 보완점 (1:20)

먼저 3차를 한 줄로 정리하면, 옛 LangChain 코드를 근거 자료를 바탕으로 1.x로 고치고, 11가지 검사와 최대 3번의 재시도로 확인하는 Streamlit 챗봇이었습니다.

챗봇 자체는 잘 동작했지만, 서비스로 보면 부족한 점이 세 가지 있었습니다.

첫째, 배포입니다. Streamlit Cloud 앱 하나에 화면, 그래프, 저장을 모두 싣다 보니 역할을 나누거나 다른 곳으로 옮기기 어려웠습니다.

둘째, 사용자 관리입니다. 비밀번호 없이 아이디만으로 로그인해서, 아이디만 알면 남의 대화를 볼 수 있었고, 대화를 지우는 기능도 없었습니다.

셋째, 평가입니다. 코드 변환만 평가했기 때문에 문서 질문에 얼마나 잘 답하는지는 알 수 없었고, 최종 검증 통과율도 53%에 머물렀습니다.

5. 해결 방향 (1:00)

그래서 4차의 방향을 두 가지로 잡았습니다.

하나는 배포 방식입니다. Streamlit 앱 하나였던 구조를 Django API 서버와 화면으로 나누고, Docker Compose로 묶어 AWS EC2에 올렸습니다. 이 과정에서 비밀번호 로그인과 대화 삭제도 넣었습니다.

다른 하나는 평가 범위입니다. 코드 변환 평가를 같은 기준으로 다시 재고, 3차에 없던 문서 질의응답 평가를 RAGAS로 더했습니다. 질문 분류가 실패한 경우도 따로 기록해서 평가에 반영했습니다.

6. 3차 → 4차, 바뀐 것만 (0:50)

바뀐 점을 표로 모았습니다. 화면은 Streamlit에서 Django 템플릿과 HTML, CSS, JavaScript로 바뀌었고, 화면에서 그래프를 직접 부르던 구조는 Django API를 거치도록 바뀌었습니다. 로그인에는 비밀번호가 추가됐고, 대화는 삭제까지 할 수 있습니다. 반대로 모델인 gpt-4o-mini와 LangGraph 파이프라인, RAG 데이터는 3차 그대로 사용했습니다.

7. 배포 구성 (1:10)

4차의 전체 구성입니다. 브라우저의 요청은 Nginx를 거쳐 Django로 가고, Django가 LangGraph 파이프라인을 부릅니다. 사용자와 대화 기록은 TiDB에, 문서 검색은 Chroma에 있고, 모델은 OpenAI의 gpt-4o-mini를 씁니다.

이렇게 구성한 이유는 세 가지입니다. Nginx까지 Docker Compose로 묶어서, 서버에서 명령 하나로 배포할 수 있게 했습니다. CSS와 JavaScript 같은 정적 파일은 Nginx가 바로 전달하고, 답변 스트리밍은 Nginx가 모아 두지 않도록 버퍼링을 껐습니다. 그리고 Gunicorn worker를 1개로 두어서, 대화 기억이 한곳에 유지되도록 했습니다.

8. Django API (1:10)

Django는 애플리케이션 두 개로 나눴습니다.

users는 사용자 관리를 맡습니다. 회원가입, 로그인, 로그아웃 세 가지 API가 있습니다.

langration은 질문을 받아 파이프라인을 돌려 답을 만들고, 대화 세션을 관리합니다. 새 대화 만들기, 대화 목록과 대화 불러오기, 대화 삭제, 그리고 LLM에 질문하는 chat API가 있습니다. chat API는 답을 한 번에 주지 않고 진행 단계를 스트리밍으로 보냅니다.

화면은 이 API만 불러서 동작하기 때문에, 화면과 서버를 따로 고칠 수 있습니다.

9. 질문 분류 (1:00)

파이프라인 중에서 4차에 바뀐 부분은 질문 분류입니다.

첫째, 3차에서는 모든 질문을 LLM이 판단했는데, 4차에서는 코드가 있는지, 옛 import나 옛 API 이름이 있는지를 코드 규칙으로 먼저 확인하고, 나머지만 LLM이 판단합니다. 둘째, 고칠 코드 없이 변환해 달라고 하면 억지로 변환하지 않고, 코드를 붙여 달라고 안내합니다. 셋째, 분류가 실패하면 일반 답변으로 넘기지 않고 따로 기록해서, 평가에서도 실패로 셉니다.

10. 진행 표시 (1:00)

답을 기다리는 동안 지금 어떤 단계인지 보여 주는 기능은 3차에도 있었습니다. 4차에서는 보여 주는 방식이 바뀌었습니다.

3차는 화면과 그래프가 한 프로그램 안에 있어서, 단계가 끝날 때마다 Streamlit이 화면을 다시 그렸습니다. 4차는 화면과 서버가 나뉘어 있기 때문에, Django가 단계가 끝날 때마다 한 줄씩 보내고, Nginx는 그걸 모아 두지 않고 바로 넘겨 줍니다. 화면은 도착한 줄을 차례로 쌓습니다. 오른쪽 화면처럼 질문 분류, 바꿀 부분 찾기, 근거 찾기, 고치기, 검사 순서로 진행 상황이 올라옵니다.

11. 시연 (2:40)

(사전 준비: 발표 전에 배포 주소에서 질문을 하나 보내 두면 첫 응답이 빨라집니다.)

그럼 실제 배포 주소에서 바로 보여 드리겠습니다.

(배포 주소 접속 → 로그인) 먼저 아이디와 비밀번호로 로그인합니다. 4차에서 추가한 비밀번호 로그인입니다.

(확인용 예제 중 옛 코드 하나 보내기) 옛 LangChain 코드를 하나 보내 보겠습니다.

(진행 상자가 쌓이는 동안) 보내자마자 진행 상자에 지금 단계가 한 줄씩 올라옵니다. 질문 분류, 바꿀 부분, 근거 검색, 고치기, 검사 순서입니다.

(답이 나오면 오른쪽 결과 패널) 답이 나오면 오른쪽 결과 패널에서 바뀌기 전과 후의 코드, 바뀐 줄마다의 근거, 검사 결과를 확인할 수 있습니다.

(왼쪽 대화 목록) 마지막으로 대화 목록입니다. 이전 대화를 다시 열 수 있고, 필요 없는 대화는 삭제할 수 있습니다.

(응답이 늦어지면) 답이 나오는 데 20초 정도 걸릴 수 있어서, 그동안 진행 상자를 같이 보시면 됩니다.

12. 결과 정리 (1:00)

결과를 정리하겠습니다. 3차와 같은 gpt-4o-mini 모델로 쟀습니다.

최종 검증 통과율이 53%에서 93%로 올랐습니다. 정답 import까지 모두 맞힌 공통 기준 통과율은 57%에서 63%가 됐습니다. 같은 문제를 일반 LLM에 주면 0%입니다. 21개 항목으로 매긴 자체 평가는 83.3점에서 91.8점으로, B에서 A가 됐습니다. 그리고 3차에는 없던 RAG 답변 평가를 더했습니다.

13. 지표별 비교 (1:00)

지표별로 보면, 연한 색이 3차, 파란색이 4차입니다.

공통 기준은 실행 기준에 정답 import까지 모두 맞힌 문항이고, 57%에서 63%가 됐습니다. 실행 기준은 문법, 옛 import 제거, import 존재, 결과값 보존처럼 실제로 돌아가는 코드인지를 본 것으로, 87%에서 93%입니다. 최종 검증은 최대 3번의 재시도 안에 검사를 통과한 비율로, 53%에서 93%로 가장 크게 올랐습니다. 1회차 검증은 재시도 없이 첫 시도에 통과한 비율이고, 43%에서 60%입니다. 네 지표 모두 3차보다 올랐습니다.

14. RAG 평가 (1:00)

마지막은 RAG 평가입니다. 3차에서는 코드 변환만 평가해서, 문서 질문에 얼마나 잘 답하는지는 알 수 없었습니다. 4차에서는 RAGAS로 문서 질의응답 30문항을 평가했습니다.

근거 충실도는 찾은 문서 내용으로만 답했는지를 봅니다. Langration은 0.89이고, 문서를 쓰지 않는 일반 LLM은 0입니다. 사실 일치는 정답과 사실이 맞는지를 보는데, 일반 LLM 0.32보다 높은 0.48이 나왔습니다.

15. 마무리 (0:20)

정리하면 4차에서는 3차 챗봇을 Django 웹 서비스로 바꿔 AWS에 배포했고, 평가를 RAG까지 넓혔습니다. 들어 주셔서 감사합니다. 질문 받겠습니다.